



PROGRAMA DE FORMACIÓN MARVELL

Diseño Digital para Sistemas DSP

Ejercicios Módulo 4: Circuitos Aritméticos I

Profesor: Del Barco, Martín.

Alumnos: Monja, Ernesto Joaquín
Zúñiga, Guillermo Rubén Darío

Índice

1. Introducción	2
2. Desarrollo	2
2.1. RCA <i>N-bit</i> Parametrizable	2
2.2. <i>Carry-Lookahead Adder</i> (CLA) Jerárquico	4
2.3. <i>CSLA</i> por bloques de 4 bits (opcional)	6
2.4. Multiplicador Secuencial <i>Shift and Add</i>	7
2.5. Array Multiplier 8×8 (combinacional)	9
2.6. Multiplicador <i>Booth Radix-2</i>	11
3. Conclusiones	12
4. Referencias	13

1. Introducción

El presente informe detalla el diseño, implementación y verificación de diversos circuitos aritméticos fundamentales para el procesamiento digital de señales (*DSP*), desarrollados en lenguaje *System Verilog*. A lo largo del documento se exploran distintas arquitecturas de sumadores —Ripple Carry Adder (*RCA*), Carry-Lookahead Adder (*CLA*) y Carry Select Adder (*CSLA*)— así como de multiplicadores —secuencial *Shift and Add, Array* combinacional y *Booth Radix-2*—. El objetivo principal es analizar y contrastar el desempeño teórico y práctico de cada topología en términos de retardo temporal, uso de área lógica, latencia y *throughput*, validando su correcto funcionamiento mediante simulaciones exhaustivas en entornos de *testbench*.

2. Desarrollo

2.1. RCA *N-bit* Parametrizable

Se implementó un Ripple Carry Adder (*RCA*) parametrizable de N bits utilizando una estructura puramente jerárquica y generativa en *System Verilog*. La arquitectura se basa en la instanciación concatenada de módulos *Full-Adder* mediante una sentencia *generate-for*. Cada bloque procesa dos bits de entrada ($a[i]$, $b[i]$) y el acarreo entrante ($c[i]$), produciendo la suma parcial ($s[i]$) y el acarreo saliente ($c[i + 1]$).

Dado que la topología encadena *adders* completos de 1 bit de forma lineal, la cantidad total de bloques *Full-Adder* requeridos escala proporcionalmente con el ancho de palabra N :

$$\text{Cantidad de FAs} = N \quad (1)$$

Para las distintas configuraciones evaluadas, la cantidad de celdas utilizadas es:

- $N = 4$: 4 *Full-Adders*
- $N = 8$: 8 *Full-Adders* (configuración por defecto)
- $N = 16$: 16 *Full-Adders*
- $N = 32$: 32 *Full-Adders*

El camino crítico del *RCA* está determinado por la propagación en serie del acarreo a lo largo de los N estadios. El retardo total de propagación t_{RCA} viene dado por la Ec. 2. [1]

$$t_{RCA} = t_{cin \rightarrow cout_FA0} + (N - 2) \cdot t_{carry_step} + t_{cin \rightarrow sum_FAN-1} \approx \mathcal{O}(N) \quad (2)$$

El peor caso se produce cuando una transición en los bits menos significativos ($a[0]$, $b[0]$ o *cin*) requiere propagar el acarreo desde la primera posición hasta la última (por ejemplo,

al sumar $0xFE_h + 0x01_h$ con $cin = 0$). Esta dependencia lineal ($\mathcal{O}(N)$) representa la principal limitación de velocidad del sumador *RCA* para anchos de palabra grandes.

A continuación se presenta en la Tab. 1, la relación entre el número de bits N y el retardo relativo estimado acumulado en la cadena de acarreo:

Ancho de palabra (N)	Cantidad de <i>FAs</i>	Retardo relativo de acarreo ($\propto N$)
4 bits	4	$4 \Delta t$
8 bits	8	$8 \Delta t$
16 bits	16	$16 \Delta t$
32 bits	32	$32 \Delta t$

Cuadro 1: Relación de retardo de propagación según el ancho de palabra N .

La verificación funcional se llevó a cabo mediante el *testbench* auto-chequeable (`tb_rca.sv`), el cual evaluó con éxito un total de 503 vectores de prueba con resolución de tiempo de $1 [ns]/1 [ps]$:

- Borde Inferior: $0 + 0 + 0 = 0$ ($cout = 0$).
- Borde Superior: $(2^N - 1) + (2^N - 1) + 0 = 2^{N+1} - 2$ ($cout = 1$).
- Generación de Carry-out: $0xFF_h + 0x01_h = 0x100_h$ ($cout = 1, s = 0$).
- Pruebas Aleatorias: 500 estimulaciones pseudo-aleatorias con $cin \in \{0, 1\}$.

La simulación concluyó exitosamente a los $5,03 [\mu s]$ sin reportar inconsistencias tal como lo muestra la Fig. 1 y la Fig. 2.

```
>>> Compilando con iverilog...
>>> Ejecutando con vvp...
VCD info: dumpfile rca.vcd opened for output.
=====
PASS: Todos los casos evaluados con exito.
=====
tb_rca.sv:66: $finish called at 5030000 (1ps)

>>> Abriendo las ondas en GTKWave...

GTKWave Analyzer v3.3.126 (w)1999-2026 BSI

[0] start time.
[5030000] end time.
```

Figura 1: Salida de consola.

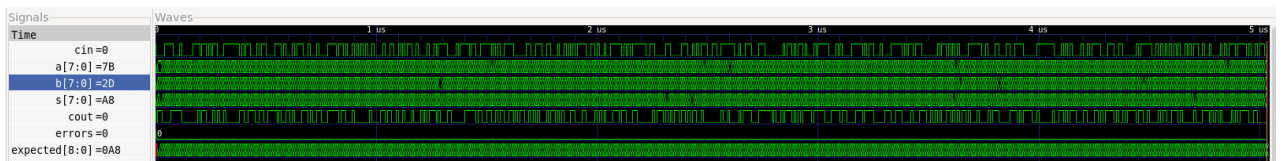


Figura 2: Salida de *GTKwave* donde se observa que la variable `errors` es siempre 0.

2.2. Carry-Lookahead Adder (CLA) Jerárquico

El *CLA* define el concepto de *generate* y *propagate*, y utilizando las Ec. 3, Ec. 4 y Ec. 5, establece el carry del bit siguiente como una relación entre el generate, propagate y el carry anterior.

$$g_i = a_i \cdot b_i \quad (3)$$

$$p_i = a_i \oplus b_i \quad (4)$$

$$c_{i+1} = g_i + p_i \cdot c_i \quad (5)$$

La arquitectura de un *CLA* de 4 bits se muestra en la Fig. 3, consiste en 4 *Full-Adders* y una lógica que procesa el generate y el propagate. Este *CLA* puede encadenarse para formar un sumador más grande, como se muestra en la Fig. 4.

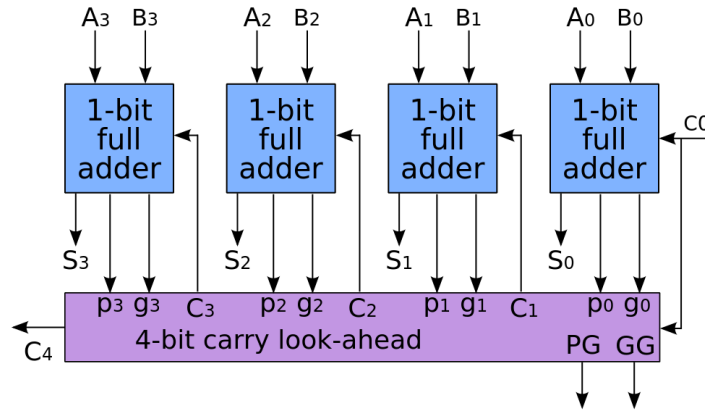


Figura 3: Arquitectura de un *CLA* de 4 bits.

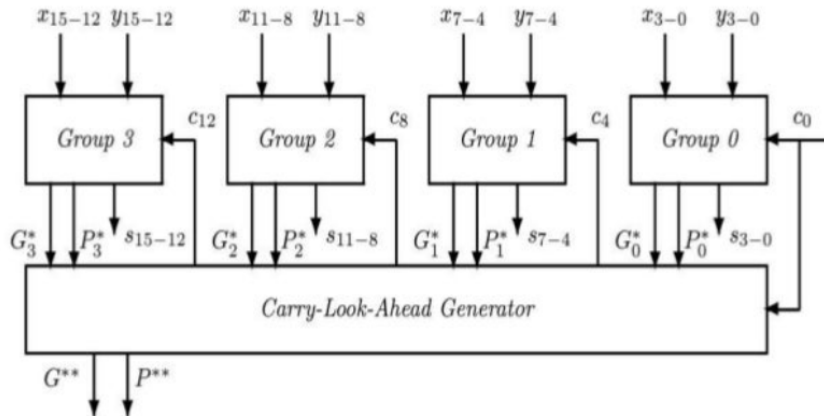


Figura 4: Arquitectura de un *CLA* de 16 bits.

Se implementó un *CLA* de 4 bits con estas ecuaciones explícitas, luego se encadenaron 4 de estos módulos para formar un *CLA* jerárquico de 16 bits. De este modo, se esperó

optimizar bastante el *delay* entre compuertas ya que se diseñó en el módulo top una lógica de *CLA* inter-bloque.

A este módulo de 16 bits se lo verificó con un testbench que inyecta determinados casos de borde y otros 1000 casos aleatorios, para luego comparar los resultados con un modelo de suma ideal. Los resultados se muestran en la Fig. 5 y la Fig. 6.

```
VCD info: dumpfile tb_cla.vcd opened for output.
-----
Errores cometidos: 0/1007
Resultado: PASS
-----
tb_cla.sv:106: $finish called at 201450 (100ps)
```

Figura 5: Resultado de la comparación con el golden model.

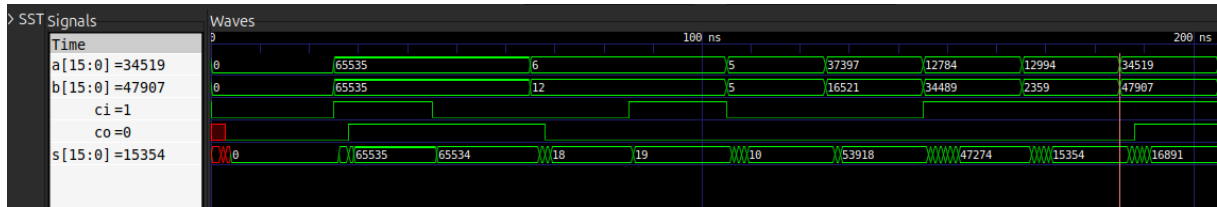


Figura 6: Porción de la forma de onda obtenida.

Se puede observar pequeñas variaciones cuando se modifica la entrada, y esto es porque se modeló un *delay* de 1 [ns] para cada compuerta lógica. Esto sirvió para luego hacer la comparación con un *RCA* de 16 bits. Si se busca el caso con el mayor *delay* (véase la Fig. 7), se observa que el máximo retardo que tiene este diseño es de 8 gates.

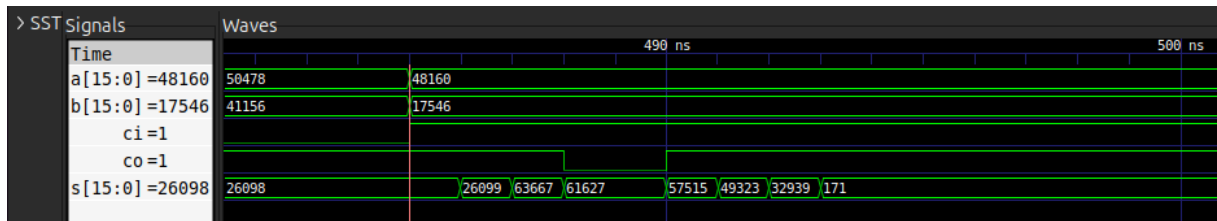


Figura 7: Forma de onda de la transición con *delay* de 8 compuertas.

Se modeló un retardo en el full adder del *RCA* de la sección anterior para realizar la comparación en el peor caso. En la Fig. 8 se puede observar un *delay* de 16 compuestas, que coincide con lo esperado teóricamente. Luego, se compararon los diseños de 4 bits, donde la Tab. 2 muestra los resultados. Para 4 bits no se observó diferencias, pero para 16 bits el *CLA* es mucho más rápido, ya que posee la lógica que procesa todos los carry en paralelo, en lugar de esperar al carry del bit anterior para realizar la suma de los bits actuales.

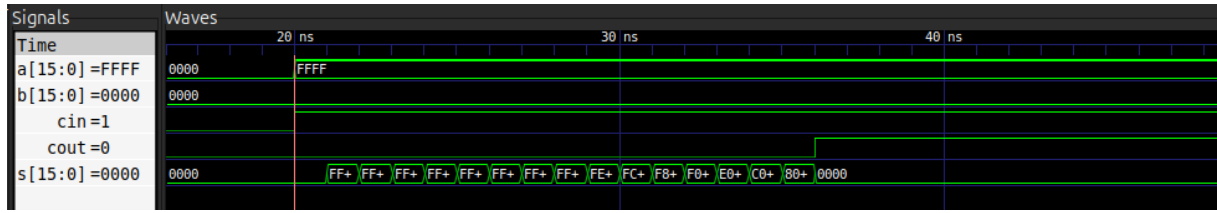


Figura 8: Forma de onda de la transición en el RCA, con *delay* de 16 compuertas.

N_{bits}	Delay RCA	Delay CLA
4	4	4
16	16	8

Cuadro 2: Comparación de retardos entre RCA y CLA (en número de compuertas).

2.3. CSLA por bloques de 4 bits (opcional)

Se diseñó un sumador Carry Select Adder (*CSLA*) de 16 bits estructurado en 4 bloques funcionales de 4 bits cada uno. El principio de funcionamiento de esta arquitectura radica en el cálculo especulativo: mientras el primer bloque computa la suma real a partir del acarreo de entrada general (*cin*), los bloques sucesivos calculan simultáneamente 2 posibles resultados (uno asumiendo $cin = 0$ y otro asumiendo $cin = 1$). Una vez que el acarreo del bloque anterior se estabiliza, se utiliza para comandar un multiplexor 2 : 1 que selecciona el resultado correcto.

El diseño se modularizó en 2 archivos principales:

- `rca4.sv`: Un sumador de acarreo propagado de 4 bits, utilizado como bloque base.
- `csla16.sv`: El módulo superior que instancia un *RCA* simple para los bits [3 : 0], y un par de *RCA* redundantes junto con multiplexores para los bloques [7 : 4], [11 : 8] y [15 : 12].

Al compilar ambos archivos, obtenemos las salidas mostradas en la [Fig. 9](#) y [Fig. 10](#).

```
>>> Compilando con iverilog...
>>> Ejecutando con vvp...
VCD info: dumpfile csla.vcd opened for output.
=====
PASS: Todos los casos evaluados con éxito.
=====
tb_csla.sv:61: $finish called at 10030000 (1ps)

>>> Abriendo las ondas en GTKWave...

GTKWave Analyzer v3.3.126 (w)1999-2026 BSI

[0] start time.
[10030000] end time.
```

Figura 9: Salida de consola.

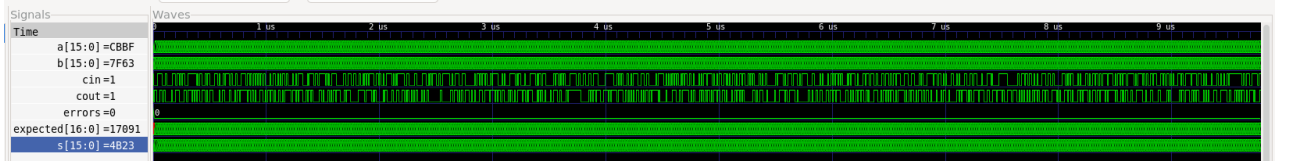


Figura 10: Salida de *GTKwave* donde se observa que la variable **errors** es siempre 0.

A continuación se presenta en la [Tab. 3](#), un análisis cualitativo del área y el retardo teórico (asumiendo compuertas ideales) para sumadores de 16 bits:

Métrica	RCA	CLA	CSLA
Área (Hardware)	Baja (≈ 16 <i>FAs</i>)	Alta (Lógica Lookahead)	Alta (≈ 28 <i>FAs</i> + <i>MUXes</i>)
Delay Teórico	$\mathcal{O}(N)$	$\mathcal{O}(\log_2 N)$	$\mathcal{O}(\sqrt{N})$ (bloques variables) / Constante por <i>MUX</i>
Camino Crítico	Ripple por 16 bits	Árbol de Lookahead	Ripple de 4 bits + 3 <i>MUXes</i>

Cuadro 3: Comparativa Teórica: CSLA, CLA y RCA (16 bits).

Si el objetivo principal de diseño es alcanzar una frecuencia máxima de operación alta (F_{max}), el *CLA* suele ser la mejor opción, ya que su retardo logarítmico escala de forma muy eficiente, aunque su implementación en celdas estándar puede requerir un *fan-in/fan-out* considerable.

Por otro lado, el *CSLA* elimina la propagación (*ripple*) inter-bloque delegando el retardo a una cadena de multiplexores rápidos, lo cual mejora drásticamente el tiempo de ciclo frente a un *RCA* puro a costa de casi duplicar el área del sumador. En tecnologías modernas donde el coste de los multiplexores es muy bajo, el *CSLA* ofrece un balance excelente entre simplicidad de diseño (reutilización de bloques *RCA*) y alta velocidad.

2.4. Multiplicador Secuencial *Shift and Add*

En sistemas donde el bajo consumo y el área es más prioritario que la latencia o el *throughput*, se utiliza una arquitectura secuencial que reutiliza sumadores y registros denominada *Shift and Add Multiplier*. El concepto de funcionamiento se muestra en la [Fig. 11](#).

Se armó un módulo en *SystemVerilog* que consiste en una *FSM* y un multiplicador *Shift and Add*. Esta máquina de estados permite controlar el multiplicador, habilitándolo solo cuando sea necesario y conociendo cuando se completó la operación, todo esto con las banderas *start* y *done*. El diagrama de estados se muestra en la [Fig. 12](#).

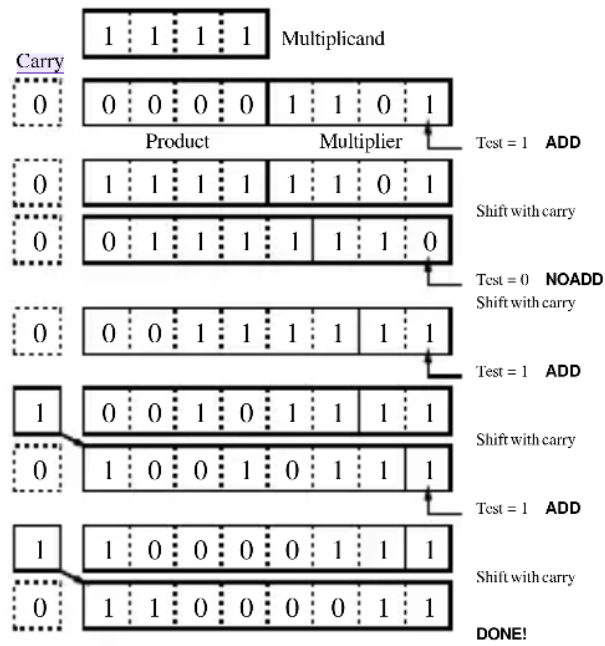


Figura 11: Ejemplo del método *Shift and Add* con 4 bits de entrada.

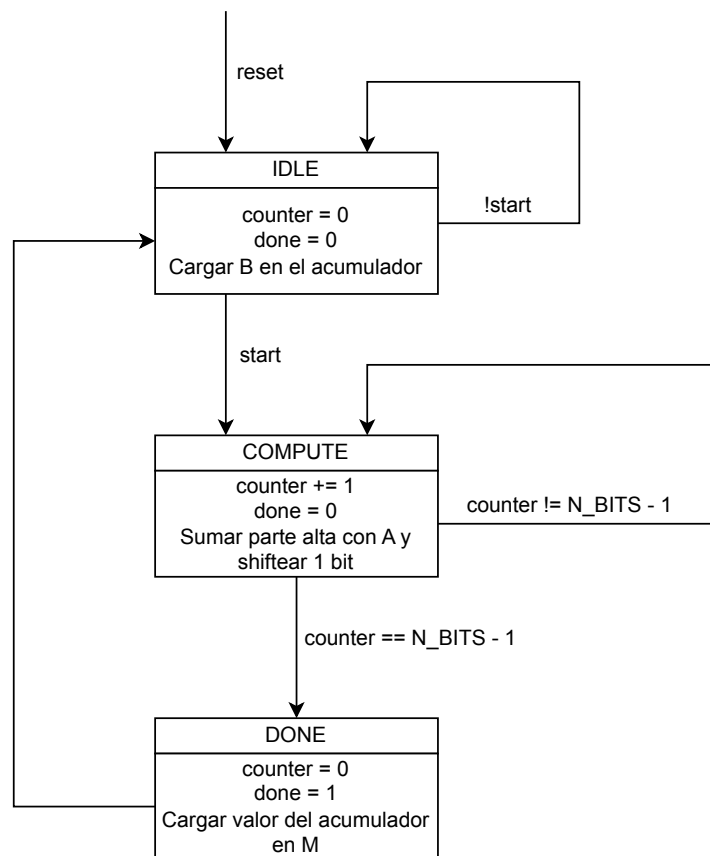


Figura 12: Diagrama de estados del multiplicador *Shift and Add*.

Este módulo se verificó en un *testbench* con poco mas de 200 casos y comparando con el resultado de la multiplicación directa. El resultado en la Fig. 13 demuestra que el módulo realiza correctamente la operación. Como se puede observar en la forma de onda de la Fig. 14, la latencia en la recepción del dato es de 10 flancos de reloj, pero el multiplicador en sí tarda 8 ciclos, como se puede observar en el contador *counter* interno. Si el multiplicador tiene latencia de 8 ciclos, entonces se puede decir que el *throughput* en este caso es de $1/8 = 0,125$.

```
VCD info: dumpfile tb_shift_add.vcd opened for output.
-----
Errores cometidos: 0/205
Resultado: PASS
-----
tb_shift_add.sv:137: $finish called at 164950 (100ps)
```

Figura 13: Resultado impreso de la simulación.

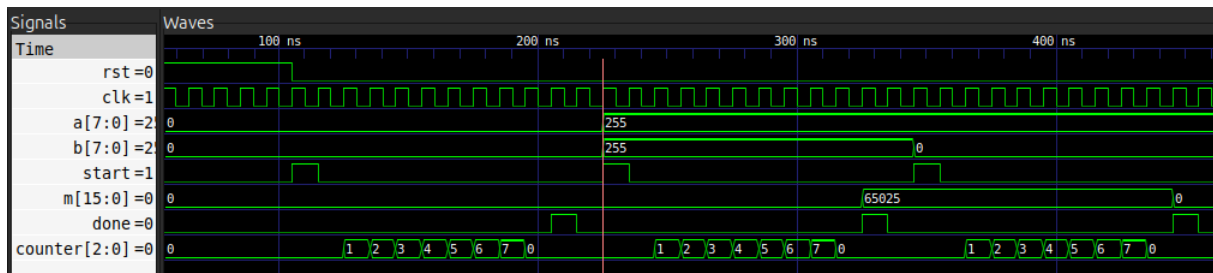


Figura 14: Formas de onda del diseño.

A comparación con otras arquitecturas, se puede ver que la cantidad de ciclos a esperar por el resultado es bastante alta, pero el uso de recursos a la hora de sintetizarlo es mucho menor. Es por esto que existen aplicaciones que requieren este método, como lo son los microcontroladores y el *IoT*.

2.5. Array Multiplier 8×8 (combinacional)

Se implementó un multiplicador unsigned combinacional de 8×8 bits utilizando una matriz de suma de acarreos conservados (*CSA*) y una fila final de propagación de acarreos (*CPA*).

La estructura del módulo se divide en tres partes principales:

- Generación de Productos Parciales (*PP*): Una matriz de 8×8 compuertas *AND* (64 compuertas en total) que generan todos los bits de productos parciales de forma paralela.
- Matriz de Reducción (*CSA*): 7 filas compuestas por 8 *Full-Adders* cada una (56 *FAs* en total). En esta etapa, los acarreos no se propagan horizontalmente dentro de la misma fila, sino que se transmiten verticalmente hacia la fila inferior.

- Fila Final de Suma (CPA): Una fila de 8 *Full-Adders* configurados como un Ripple Carry Adder (RCA) para sumar los bits acumulados y los acarreos restantes de la matriz CSA, entregando el resultado final de 16 bits.

El módulo fue verificado exhaustivamente mediante el testbench `tb_array_mul.sv`, evaluando las $256 \times 256 = 65,536$ combinaciones posibles de las entradas. La simulación completó sin errores de coincidencia respecto al resultado esperado del *host* tal como muestran la Fig. 15 y la Fig. 16.

```
>>> Compilando con iverilog...
>>> Ejecutando con vvp...
VCD info: dumpfile tb_array_mul.vcd opened for output.
=====
Inicio Test Exhaustivo (65,536 combinaciones)...
=====
RESULTADO: PASS - 65,536/65,536 vectores validados.
=====
tb_array_mul.sv:47: $finish called at 65536000 (1ps)
>>> Abriendo las ondas en GTKWave...
```

Figura 15: Salida de consola.

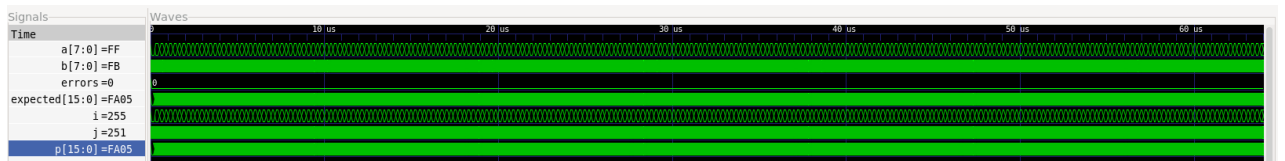


Figura 16: Salida de *GTKwave* donde se observa que la variable `errors` es siempre 0.

El camino crítico (T_{crit}) está determinado por la propagación de las señales a través del arreglo: el retardo de una compuerta *AND* inicial, la trayectoria diagonal/vertical a lo largo de las 7 etapas *CSA*, y la propagación horizontal completa en el sumador final (*CPA*) de 8 bits esta dada según la Ec. 6. [2]

$$T_{crit} = \tau_{AND} + (N - 1) \cdot \tau_{FA} + N \cdot \tau_{FA_carry} = \tau_{AND} + 15 \cdot \tau_{FA} \quad (6)$$

En total, la solución implementa **64 Full Adders** (56 en la matriz *CSA* + 8 en la fila final *CPA*) y **64 compuertas AND**. En la Tab. 4 se sintetiza la comparación de este diseño frente a la alternativa secuencial del Ejercicio 4 desarrollado en la Subsección 2.4 y la inferencia directa en *RTL*.

Podemos entonces realizar la siguiente comparación:

- Array vs. Secuencial: El multiplicador matricial maximiza el *throughput* procesando una operación completa por pulso sin requerir registros ni señal de reloj. No obstante, el costo en área aumenta significativamente al requerir 64 *FAs* en comparación con el único sumador de 8 bits.

- Array vs. RTL (*): Aunque el multiplicador matricial elimina el ciclo de reloj, su retardo combinacional crece de forma lineal $\mathcal{O}(N)$ debido a la propagación del acarreo. Al inferir el operador $*$, la herramienta de síntesis suele utilizar estructuras optimizadas como árboles de *Wallace/Dadda* con retardo logarítmico $\mathcal{O}(\log_2 N)$ o mapear directamente el cálculo a bloques *DSP* nativos de la *FPGA*, logrando una frecuencia máxima (f_{max}) superior.

Métrica	Secuencial	Array	RTL Directo (a * b)
Latencia	8 ciclos de reloj	0 ciclos (Combinacional)	0 ciclos (Combinacional)
Throughput	1 operación cada 8 ciclos	1 operación cada ciclo	1 operación cada ciclo
Recursos Lógicos	1 Adder 8b + FSM	64 FAs + 64 ANDs	Bloques DSP / LUTs Optimizadas
Camino Crítico	$\tau_{Adder8} + \tau_{FSM}$	$\tau_{AND} + 15 \cdot \tau_{FA}$	Árbol de reducción ($\sim \log_2 N$)

Cuadro 4: Comparativa entre arquitecturas de multiplicación de 8×8 bits.

2.6. Multiplicador *Booth Radix-2*

Con la recodificación *Booth* es posible reducir la cantidad de productos parciales o filas en una multiplicación, evaluando de a grupos los bits del multiplicador en lugar de individualmente. En este caso se realizó un multiplicador *Booth radix-2*, donde la cantidad de filas es la misma pero no es necesaria una lógica de extensión de signo y el *switching* en cadenas de “1” consecutivos es menor.

La codificación se muestra en la Tab. 5. Si los bits a evaluar son del mismo valor, el producto parcial correspondiente es 0, si no, tiene el valor del multiplicando positivo o negativo; al final, todos los productos parciales se suman.

$\{b_i, b_{i-1}\}$	Prod. parcial i
00	0
01	$A \ll i$
10	$-A \ll i$
11	0

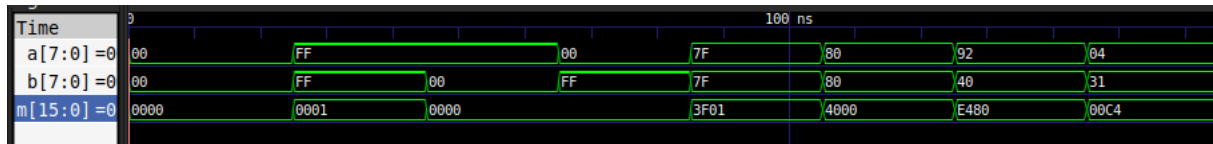
Cuadro 5: Codificación *Booth radix-2*.

En *SystemVerilog* se realizó un multiplicador con este método y se lo puso a prueba con 1000 casos de prueba, incluyendo entradas con todos los bits en “0”, todos en “1”, y con el mayor valor negativo y positivo. El resultado se muestra en la Fig. 17, indicando

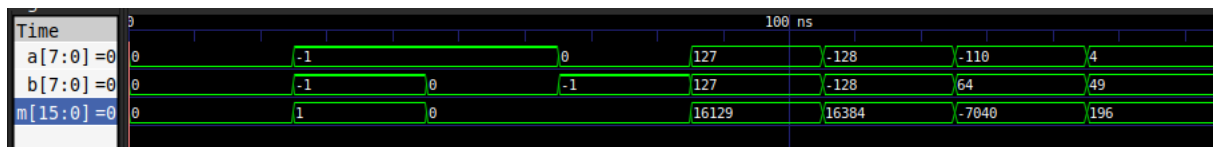
que coincidieron todos los casos con el ideal. Las formas de onda se pueden observar en la Fig. 18.

```
VCD info: dumpfile tb_booth.vcd opened for output.
-----
Errores cometidos: 0/1006
Resultado: PASS
-----
tb_booth.sv:84: $finish called at 201250 (100ps)
```

Figura 17: Resultado impreso del multiplicador *Booth*.



(a) En hexadecimal.



(b) En decimal con signo.

Figura 18: Formas de onda del multiplicador *Booth radix-2*.

Como se mencionó anteriormente, la cantidad de productos parciales a utilizar no cambia en este método comparado con un *array multiplier*, pero el tiempo de propagación sí puede ser menor, ya que no se toman en cuenta los carries en las sumas. De todas formas, se puede realizar el método *Booth radix-4* donde se evalúa el multiplicador cada 3 bits, permitiendo reducir la cantidad de filas a $N_{bits}/2$. Esto no quiere decir que este método sea el doble de rápido respecto del radix-2, ya que requiere lógica adicional para la decodificación de los 3 bits y los desplazamientos adicionales.

3. Conclusiones

El desarrollo de estos ejercicios evidencia los *trade-offs* entre área, velocidad y eficiencia en el diseño lógico digital. Mientras que las arquitecturas puramente jerárquicas o secuenciales, como el *RCA* o el multiplicador *Shift and Add*, resultan sumamente eficientes en el uso de recursos de *hardware* (siendo ideales para sistemas de bajo consumo), topologías más avanzadas como el *CLA*, *CSLA* o los arreglos combinacionales logran reducir significativamente el camino crítico y maximizar el *throughput* a costa de una mayor complejidad y área. De este modo, queda demostrado que no existe una única solución ideal, sino que la elección de la estructura aritmética dependerá estrictamente de las restricciones de frecuencia máxima y recursos disponibles que exija la aplicación final.

4. Referencias

- [1] D. M. Harris y S. L. Harris, *Digital Design and Computer Architecture*, 2nd. Waltham, MA: Morgan Kaufmann, 2012, ISBN: 978-0123944245.
- [2] B. Parhami, *Computer Arithmetic: Algorithms and Hardware Designs*, 2nd. New York, NY: Oxford University Press, 2010.
- [3] E. Monja y G. R. D. Zúñiga, *Módulo 4 - Ejercicios*, Repositorio en GitHub, Diseño Digital para Sistemas DSP, Archivos desarrollados para los ejercicios del Módulo N°4, 2026. dirección: https://github.com/ErnestMonja/Diseno_Digital_para_Sistemas_DSP/tree/main/M%C3%B3dulo%204%20-%20Ejercicios